

RankKnot

A 208-Byte In-Memory Rank Summary for Deferred Tensor Analysis

Observe every sample now; choose quantiles, groups, diagnostics, and quantization later

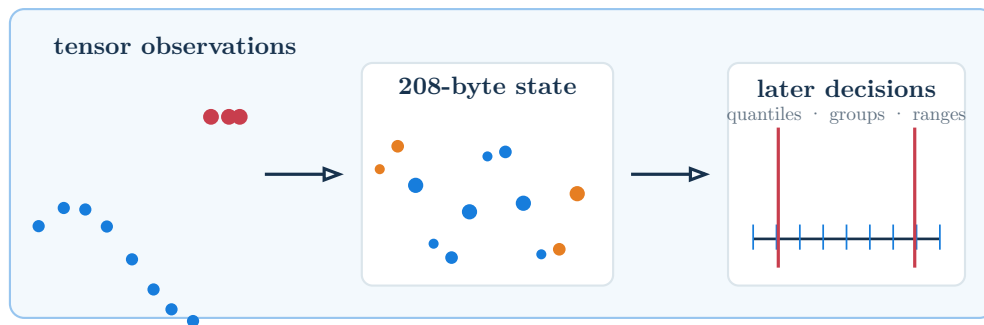
tensor histories

rank transport

exact ties

cold decisions

monatq implementation note · version 0.4



The current implementation `TensorDigest<f32, RankKnot>` is an optional statically selected kernel. Thirty-two value knots, 16-bit probability masses, one tie mask, and exact extrema occupy 208 bytes of state per position. The live collector also uses a 256-row input buffer by default and worker-local flush scratch. The generic API includes merge, analysis, zero-filtering, serialization, and optional visualization methods, but RankKnot's implementations are explicit stubs that panic; direct support access and quantization remain future work.

An important wording distinction RankKnot incorporates every supported observation that reaches a successful flush, but it does not retain raw values afterward. It preserves an approximate empirical distribution from which later rank-based decisions can be made. Raw samples, their temporal order, and cross-position relationships cannot be reconstructed. NaN input is unsupported and currently fails during flush rather than being rejected atomically by update.

Contents

1	Executive summary	2
2	Observe now, decide later	3
2.1	Why fixed summaries are often too early	3
2.2	What cannot be deferred	4
3	A compact empirical distribution	4
3.1	Positive weighted support	4
3.2	The 208-byte K32 summary-state layout	4
4	Streaming compression	5
4.1	Buffer first, compress rarely	5
4.2	Tail-companded rank cells	5
4.3	Mixed cells use weighted means	5
4.4	Prefix-rounded probability	6
5	Cold reconstruction and regrouping	6
5.1	A monotone quantile curve	6
5.2	Choose robust ranges later	7
5.3	Future consumer: regrouping	7
6	Future consumer: post-training quantization	7
6.1	Evaluate an affine quantizer	7
6.2	Search candidates on the stored measure	8
7	Accuracy measurement	8
7.1	Ties require rank intervals	8
7.2	Historical K64 Python feasibility protocol	8
7.3	Historical K64 general-distribution results	9
7.4	Historical K64 discrete observations	9
7.5	Current Rust K32 evidence	9
7.6	Important caveats	10
8	A deliberately small systems design	10
8.1	Rust representation	10
8.2	One current ingestion reducer	11
8.3	Minimal configuration	11
9	Validation status	11
9.1	Implemented coverage	11
9.2	Open work	11
10	Risks and limitations	12
11	Reproducibility	12
12	Conclusion	13
	References	13

1 Executive summary

`TensorDigest` receives full tensor samples and tracks one distribution at every flat tensor position. `TensorDigest<f32, RankKnot>` is now an implemented, statically selected backend optimized for update throughput and compact state. It buffers complete samples, compresses each position independently in parallel, and exposes per-position quantiles and exact extrema.

RankKnot separates **observation** from **decision**, but today it covers only recording and rank queries. Generic `TensorDigest` methods provide construction, shape, weight, update, flush, quantile, merge, analysis, zero filtering, serialization, and optional visualization operations; RankKnot adds sample count, configuration, and tensor/cell extrema. Its unsupported

generic operations currently panic as unimplemented. Direct knot access, working grouping, persistence, visualization, analysis, and PTQ search remain design directions.



Figure 1: The current recorder. Every supported tensor observation updates the same compact state. Quantiles and extrema are available now; richer cold consumers require future APIs.

The K32 state uses 208 bytes per tensor position: 32 `f32` values, 32 `u16` masses, a 64-bit purity mask, and two `f32` extrema. This is about one twenty-fourth of the repository t-digest’s 4,900-byte summary state and 160 bytes smaller than Quantile Spine’s 368-byte state. Equal values share the ordinary knot budget, and extrema remain exact even though interior observations are compressed.

Memory scope The 208-byte figure is summary state, not complete live memory. With `f32` inputs, the default 256-row buffer adds 1,024 bytes per position, for approximately 1,232 tensor-scaled retained bytes per position. Worker-local vectors and object headers add a small non-tensor-scaled overhead and appear in peak allocator measurements.

Property	t-digest	Quantile Spine	RankKnot K32
Summary state/position	4,900 B	368 B	208 B
<code>f32</code> input buffer/position	800 B	1,024 B	1,024 B
Additional tensor sort buffer	none	1,024 B	none
Approx. tensor-scaled retained total	5,700 B	2,416 B	1,232 B
State primitive	weighted centroids	rank anchors	up to 32 weighted knots
Repeated values	interpolated	anchor dependent	retained pure interval
Exact min/max	yes	yes	yes
Public <code>merge_*</code> contract	implemented	stub: panics	stub: panics

The table assumes default configurations, `f32`, and a 64-bit target. Allocator instrumentation over 32 positions measured RankKnot at 39,432 live bytes and 45,064 peak bytes, versus 182,408 live bytes for t-digest and 77,320 for Quantile Spine. The component totals explain the scaling; exact allocator totals also include shape vectors, headers, and worker scratch.

Evidence status Current Rust K32 accuracy and memory are measured by `backend_accuracy`, and the repository benchmark exercises update throughput. A local Apple M4 run is reported below. The broader checked-in Python study remains historical K64 feasibility evidence, not a measurement of the current K32 backend. Million-row drift and downstream application quality remain open.

2 Observe now, decide later

2.1 Why fixed summaries are often too early

A minimum and maximum preserve endpoints but reveal nothing about how probability is distributed between them. A fixed histogram commits to bin edges before the eventual query is known. A single percentile commits to one tail policy. A preselected channel grouping prevents later comparison with per-tensor or hardware-aligned alternatives.

RankKnot instead records a reusable approximation of rank mass. The current API supports medians, arbitrary quantiles, percentile bands assembled by the caller, and exact per-position extrema. The encoded representation could also support grouping, distribution plots, saturation diagnostics, alert thresholds, and PTQ clipping if dedicated consumers or public state access are added.

API status One generic `TensorDigest` contract now includes merge, analysis, zero filtering, serialization, and optional visualization. T-digest implements those operations; RankKnot and Quantile Spine expose them through explicit `unimplemented!` stubs that panic. RankKnot state remains crate-private, and callers cannot enumerate knots or hand weighted support directly to a quantizer.

The architecture is closer to a compact tensor history than to a quantizer, but today only its per-position rank-query surface is implemented.

2.2 What cannot be deferred

No fixed-memory summary retains raw sample identity, temporal order, correlations between positions, or exact arbitrary quantiles. If a later analysis needs sample-level replay, covariance, or causality, it must store different information. Comparison-based quantile sketches necessarily trade memory for error [1]. RankKnot targets representative tensor distributions rather than a universal worst-case guarantee.

3 A compact empirical distribution

3.1 Positive weighted support

An observed stream defines an empirical distribution made of point masses. RankKnot replaces that large measure with at most 32 positive weighted locations. In one dimension, this can be understood as moving nearby rank mass onto representative support points—a small transport coreset [2].

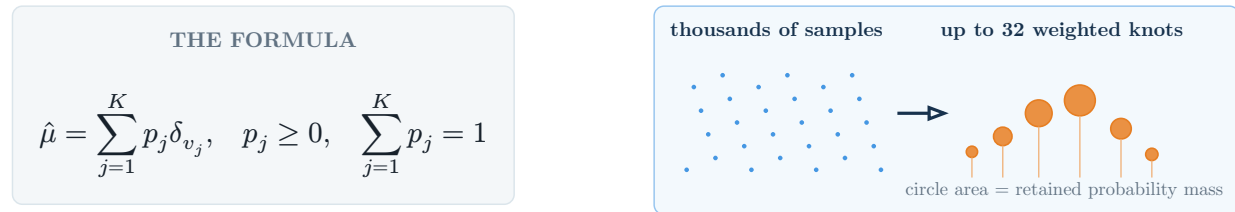


Figure 2: **The compressed history.** Thousands of observations become a short weighted support. Large circles carry more probability; positivity and normalization are explicit.

Every input affects the state, but most inputs cease to be individually identifiable after compression. Any number of distinct values may enter. At most 32 locations remain exact; excess support is approximated by the same rule used for continuous data. There is no separate “number of atoms supported” configuration.

3.2 The 208-byte K32 summary-state layout

All positions receive one observation at the same logical tensor update, so one `u64` sample count lives in RankKnot storage rather than in every position. The per-position state spends its final eight bytes on exact extrema. Input buffering and flush workspace are separate from this layout. NaN input is unsupported: `update` currently buffers it without validation, and a later flush panics in the sort comparator. This is not an atomic whole-update rejection contract.



Figure 3: **Summary-state accounting.** The four terms are `f32` locations, `u16` masses, a 64-bit tie mask, and two `f32` extrema. Zero mass identifies an unused slot.

Field	Bytes	Meaning
values[32]: f32	128	mixed-group means or retained exact values
masses[32]: u16	64	probability quanta; active masses sum to 65,535
pure_mask: u64	8	one bit per active location; upper 32 bits are unused
min, max: f32	8	exact endpoints, including supported infinities
Summary-state total	208	no pointers and no per-position allocation

Per-position missingness would break the shared-count assumption and must pay an explicit memory or semantic cost. Atomic NaN rejection would require an ingestion validation pass; the current performance-oriented contract instead assumes NaN-free observations. Positive and negative infinity are supported and protected as pure singleton groups.

4 Streaming compression

4.1 Buffer first, compress rarely

Updates append full row-major tensor samples to one input buffer whose public `buffer_capacity` defaults to 256. A flush handles positions independently using Rayon, with a minimum parallel chunk length of 64:

1. Gather one position into a worker-local `Vec<f32>` and unstable-sort it with `partial_cmp`.
2. Linearly merge it with up to 32 old representatives, coalescing equal values during the merge.
3. Evaluate 31 fixed arcsine cuts and snap each to the nearest entry boundary.
4. Skip duplicate or unusable cuts; fewer than 32 groups may remain. Protect infinities as pure singleton groups.
5. Store exact pure singletons and `f64`-accumulated weighted means, rounded to `f32`, for mixed groups.
6. Prefix-round cumulative masses to 65,535 with ties-to-even, difference adjacent prefixes, and update extrema.

At the default capacity the merged stream contains at most 288 entries. Scratch consists of two preallocated worker-local vectors and one fixed 31-element boundary array. It scales with parallel work rather than tensor width; there is no `numel × batch_rows` sorted copy, per-position hash table, or iterative optimizer. The vectors do allocate when a Rayon job initializes, so “no dynamic allocation” applies only to persistent per-position state, not flush workspace.

4.2 Tail-companded rank cells

Uniform rank cells spend the same resolution at the median and at the extremes. Many later decisions—robust ranges, alarms, saturation estimates, and quantization—care about tails. RankKnot therefore places fixed slots uniformly in an abstract coordinate that bends toward probability zero and one.

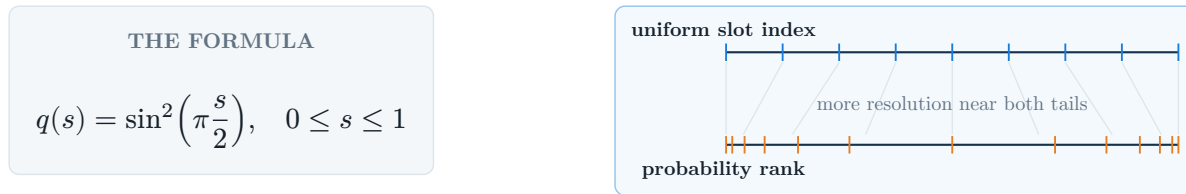


Figure 4: **Tail-aware rank placement.** Equal slot steps on the upper line become denser at both ends of probability space. This is the same broad tail-allocation principle that benefits t-digest [3].

A large exact tie may contain several desired cuts. Duplicate snapped cuts are skipped, so a 50% zero mass consumes one support location rather than half the state. The current implementation does not recursively refill unused cuts; a tie-heavy stream can therefore finish with fewer than 32 active locations.

4.3 Mixed cells use weighted means

A cell containing several locations stores their weighted mean. It is the constant representative minimizing squared value reconstruction error, and masses plus first moments combine cleanly before the next lossy repartition.

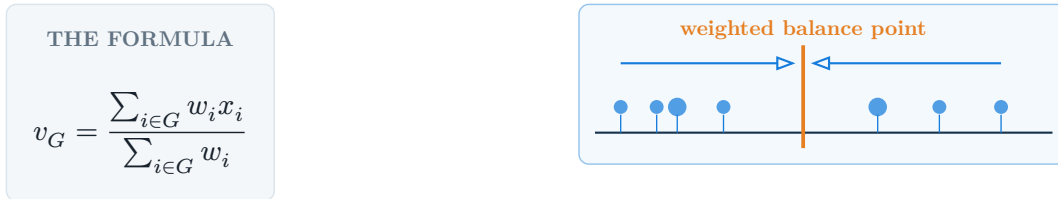


Figure 5: **One representative per mixed cell.** The orange line is the weighted balance point. In the feasibility prototype, medians produced materially larger smooth-distribution error under cold interpolation.

A cell containing one exact location retains that location and sets its purity bit. A mixed cell remains mixed even if its rounded mean happens to equal an input value, preventing accidental invention of a tie.

4.4 Prefix-rounded probability

Sixteen-bit probability masses keep the K32 state compact. The exact total sample count remains outside each position. Cumulative prefixes are rounded first, then differenced, so every represented CDF boundary is directly controlled.

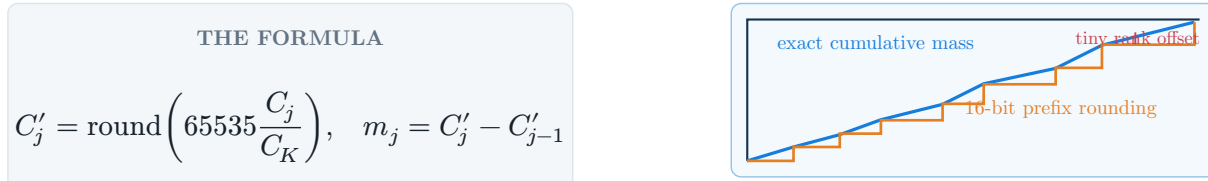


Figure 6: **Sixteen-bit mass encoding.** Each orange step follows the blue cumulative mass. Prefix rounding avoids independent mass errors all drifting in one direction.

The current boundary discrepancy is at most one half probability quantum. Repeated compression and requantization can still accumulate approximation; long streams and repeated distribution shifts remain mandatory tests.

5 Cold reconstruction and regrouping

5.1 A monotone quantile curve

Each mixed knot is anchored at the center rank of its encoded mass. Neighboring mixed anchors are joined linearly. A pure knot contributes a horizontal interval across its mass. Sorting values and ranks makes the resulting quantile curve monotone.

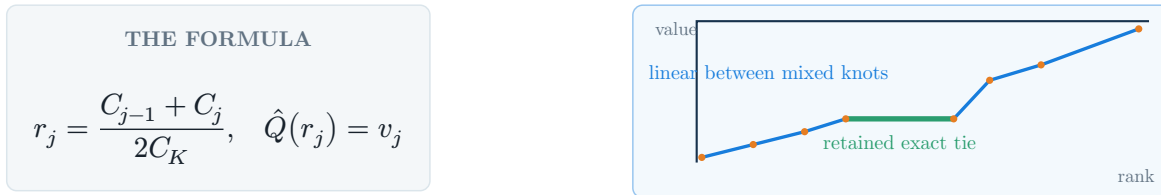


Figure 7: **Decode only when needed.** Blue segments interpolate unresolved continuous mass. The green plateau represents an exact retained tie across a whole rank interval.

Endpoint requests bypass this interpolation. For a nonempty summary, any $q \leq 0$ returns exact `min`, any $q \geq 1$ returns exact `max`, and a NaN probability returns NaN. An empty summary returns `0.0` for every probability, including NaN, because the emptiness check runs first. Queries flush pending rows first and can therefore trigger the unsupported-input panic if buffered observations contain NaN. Tensor-wide queries run in parallel; `cell_quantiles` evaluates one position locally. When an infinity is adjacent to a finite anchor, the interpolation helper returns the infinity throughout that gap rather than producing NaN.

5.2 Choose robust ranges later

A caller can defer its tail policy until after collection. One consumer may want a central visualization band; another may choose an anomaly threshold; another may estimate a clipping interval. All can use different tail probabilities from the same state.

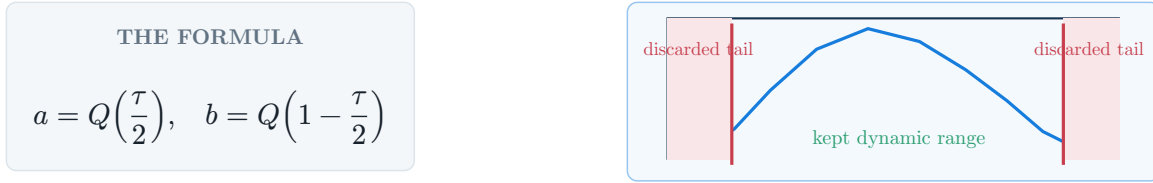


Figure 8: **A deferred central range.** The shaded total tail mass is selected later. Collection does not hardcode τ , symmetry, or the eventual use of the range.

5.3 Future consumer: regrouping

The generic contract now exposes `merge_cells`, `merge_channels`, and `merge_all`, but RankKnot's implementations are deliberate `unimplemented!` stubs and panic when called. Its encoded state is crate-private, so callers cannot provide the missing union externally. A working implementation could treat each stored object as a positive weighted measure: scale masses by observation counts and optional group weights, union support, sort once, and run a compatible compressor.



Figure 9: **Deferred grouping.** Fine-grained histories become one group distribution without replaying the original tensor samples. Grouping policy is a cold decision.

Aggregation latency would not be a performance requirement. Such a future cold merge could allocate temporary vectors and globally sort all input support. The present reducer is used only for buffered ingestion.

6 Future consumer: post-training quantization

PTQ illustrates why deferred distribution storage could be useful. A calibration run could collect activation histories before choosing bit width, symmetric versus asymmetric ranges, per-tensor versus per-channel grouping, or clipping policy. No current RankKnot API exposes weighted knots to a quantizer, and no quantizer consumes this state; the following equations describe a proposed consumer. PTQ methods commonly use a small representative set to establish activation ranges [4], [5].

6.1 Evaluate an affine quantizer

A bounded real interval determines the step spacing of a finite integer codebook [6], [7]. Wider endpoints preserve outliers but make every interior step coarser.

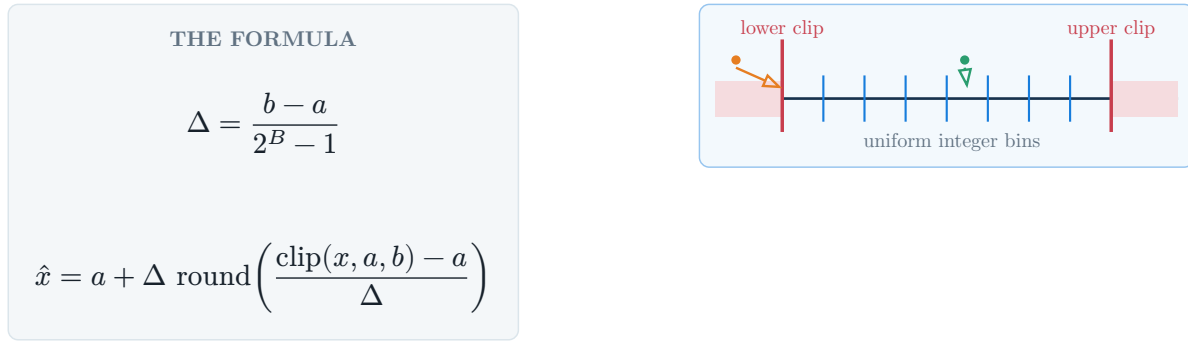


Figure 10: **One possible deferred quantizer.** The red endpoints and bit width are not chosen while observations are collected. They are candidate decisions evaluated afterward.

6.2 Search candidates on the stored measure

With a future support-export or dedicated PTQ API, a cold quantizer could evaluate candidate intervals directly on up to 32 weighted locations. This would approximate activation reconstruction error while making search cost independent of the original sample count.

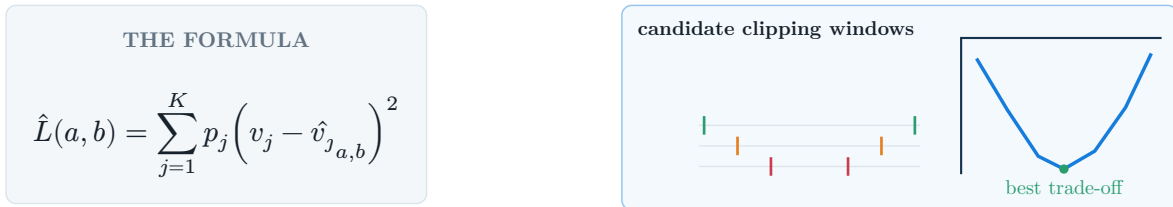


Figure 11: **Quantization is a consumer, not the state.** Narrow, medium, and wide candidate windows trade clipping against rounding. Other consumers can ignore this objective entirely.

This marginal objective does not model Hessians, cross-channel correlations, or final task loss. More sophisticated quantization methods can consume the same distributional evidence alongside their own model-specific information.

7 Accuracy measurement

7.1 Ties require rank intervals

A requested quantile inside an empirical jump is correctly answered by the repeated value. Measuring only one side of the CDF would report an error even though the returned value is a valid generalized quantile [8].

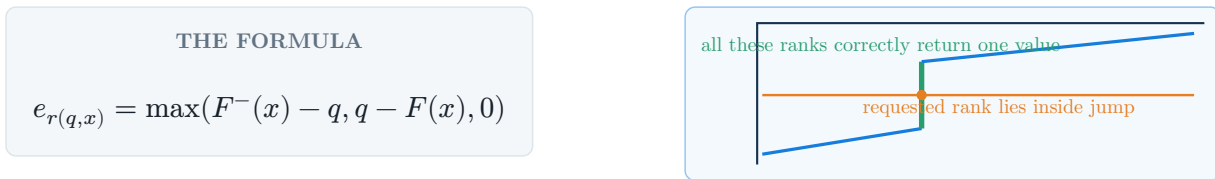


Figure 12: **Tie-aware rank error.** The orange requested rank lies inside the green CDF jump, so returning the repeated value has zero error.

Application-specific consumers must report their own outcomes as well. Rank error evaluates the recorder; it does not replace downstream quality, reconstruction error, alert precision, or visualization usefulness.

7.2 Historical K64 Python feasibility protocol

The checked-in historical experiment uses two held-out seeds, 32,768 f32 observations per workload, 256-row K64 prototype flushes, and 235 query probabilities from one ten-thousandth through its upper complement. Parameter exploration used a separate seed. The Python prototype simulates a **64-knot** state with 16-bit prefix masses and persistent f32 representatives.

Repository **TDigest** and **QuantileSpine** run through a temporary Rust crate on identical ordered values using their own default buffering. These measurements do not describe the current K32 Rust implementation.

Workloads include uniform, normal, two lognormal scales, Student-t, shuffled and blocked mixtures, one and five added atoms, 50% zero, constants, 8–256-level quantization, and identical uniform multisets in random, ascending, and descending order. Ordered cases are diagnostics rather than proposed gates.

7.3 Historical K64 general-distribution results

Workload	K64 mean	t-digest mean	K64 max	t-digest max
uniform	0.000267	0.001414	0.001150	0.005179
normal	0.000271	0.001018	0.001069	0.005746
lognormal, $\sigma=1$	0.000346	0.002420	0.001304	0.008954
lognormal, $\sigma=2$	0.000584	0.004453	0.001637	0.011169
Student-t, $df=2$	0.000392	0.000861	0.001288	0.004347
bimodal, shuffled	0.000927	0.009599	0.013983	0.066852
bimodal, blocked	0.001288	0.010447	0.016979	0.066882

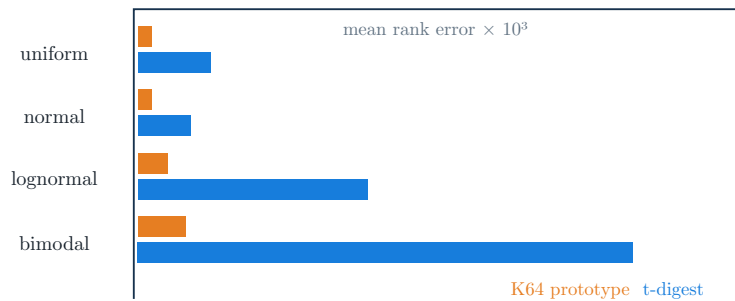


Figure 13: Historical K64 Python mean valid-rank error on selected held-out workloads. Bars use thousandths of rank. Lower is better; separated modes remain harder than smooth distributions.

Across all 36 workload/seed pairs, the historical K64 prototype’s mean-of-means was 0.000330 and its worst observed maximum was 0.016979. The repository t-digest values were 0.006928 and 0.246068. K64 had lower mean and maximum error on every nonconstant pair; both methods tied on constants. This is capacity-design evidence, not a claim about current K32.

7.4 Historical K64 discrete observations

Workload	Mean error	Worst error	Interpretation
five added atoms	0.000117	0.001653	ties share the ordinary knot budget
8 levels	0	0	all levels retained
16 levels	0	0	all levels retained
32 levels	0	0	all levels retained
64 levels	0	0	all levels retained in this test
256 levels	0.000831	0.002344	excess levels interpolated

Zero means zero at the 235 tested ranks, not equality at every possible probability. The historical result demonstrates that repeated values need no separately hardcoded atom capacity.

7.5 Current Rust K32 evidence

The current `backend_accuracy` executable uses 100,000 samples at each of 32 representative tensor positions, nine query probabilities from 0.001 through 0.999, and tie-aware empirical rank intervals. RankKnot and Quantile Spine use their default 256-row buffers; t-digest uses its default 200-row buffer. Its adversarial suite uses 65,536 samples at one position and

1,003 probabilities. Heap figures come from an instrumented global allocator and exclude inputs, exact truth, and query outputs.

Workload	K32 mean / max	t-digest mean / max	Spine mean / max
normal	0.000458 / 0.001460	0.000627 / 0.006950	0.000302 / 0.001490
uniform	0.000681 / 0.001170	0.001334 / 0.007380	0.000575 / 0.001780
lognormal	0.000860 / 0.002290	0.001831 / 0.012570	0.000758 / 0.002130
32-level normal	0.000094 / 0.000660	0.009511 / 0.034780	0.011590 / 0.055840
50% zeros	0.000380 / 0.001380	0.026949 / 0.252720	0.000096 / 0.000600
95% zeros	0.000508 / 0.006160	0.000359 / 0.004750	0.000031 / 0.000420
heterogeneous	0.000511 / 0.002260	0.003170 / 0.051750	0.001411 / 0.039770

K32 substantially improves on t-digest for most representative workloads, especially quantized and 50%-zero data, but it is not uniformly best. Quantile Spine has lower mean error on several smooth distributions and clearly wins on 95%-zero activations. K32 often has the smallest maximum error and is particularly strong on the heterogeneous workload. Adversarial results are mixed, so no universal ordering is claimed.

Allocator measurements at 32 positions were 39,432 live / 45,064 peak bytes for K32, 182,408 / typically 239,208 for t-digest, and 77,320 / 113,160 for Quantile Spine. At one position K32 measured 1,240 live / 6,872 peak bytes; the difference between live and peak is worker-local ingestion scratch.

A local Apple M4 Divan run with RankKnot’s default capacity 256 and each comparison backend’s default configuration measured median update times of 8.73 ms for K32 versus 7.72 ms for t-digest on $64 \times 64 \times 1,000$ normal data, 8.81 versus 7.51 ms on the corresponding uniform data, and 21.47 versus 26.81 ms on $256 \times 256 \times 200$ uniform data. K32 therefore remained 13–17% slower on the smaller repeated-flush cases but was about 20% faster on the larger one. These are platform-specific observations, not portable guarantees.

7.6 Important caveats

- **Historical extreme tails:** in the K64 study, t-digest was often two to three times more accurate at individual smooth tail queries, although K64’s absolute tail errors were small.
- **Separated modes:** linear interpolation across a large unsupported value gap remains a principal failure mode.
- **Extrema:** current K32 endpoint requests use exact min/max sidecars; the historical K64 interior results did not include endpoint requests.
- **Throughput scope:** the Apple M4 timings above are local medians. Different tensor widths, Rayon pools, CPUs, and system load can change the comparison.
- **No consumer evidence:** no RankKnot quantizer, group merge, alert, visualization, or serialization study exists because those APIs are not implemented.

8 A deliberately small systems design

8.1 Rust representation

```
const RANK_KNOT_K: usize = 32;
```

```
#[repr(C)]
struct RankKnotState {
    values: [f32; RANK_KNOT_K],
    masses: [u16; RANK_KNOT_K],
    pure_mask: u64,
    min: f32,
    max: f32,
}
```

`RankKnotState` is crate-private; the public `RankKnot` type is a zero-sized kernel marker. One `u64` sample count is shared across positions in `RankKnotStorage`. Temporary weights and weighted moments use `u64` and `f64`; persistent representatives round to `f32`. With 256 buffered rows, tensor-scaled retained storage is approximately $208 + 1,024 = 1,232$ bytes per position. Worker scratch and object headers are additional.

8.2 One current ingestion reducer

The deterministic reducer currently implements buffered ingestion only. Reusing it for cold merges remains a design direction. Current `RankKnot` callers cannot access the encoded support, and no `RankKnot`-specific cache or query-time workspace is retained per position.

8.3 Minimal configuration

Item	Current value	Status
<code>buffer_capacity</code>	256	public and configurable; must be positive
input type	<code>f32</code>	<code>RankKnot</code> is implemented only for <code>f32</code>
knot count	32	fixed internal constant
mass quanta	65,535	fixed internal <code>u16</code> normalization
rank scale	arcsine	fixed internal targets
NaN	unsupported	not validated on update; panics during flush
infinities	supported	protected pure singleton groups and exact endpoints

Only `buffer_capacity` is public configuration. The mass encoding, knot count, and scale function remain implementation details while long-stream and cross-platform behavior stabilizes.

9 Validation status

9.1 Implemented coverage

Area	Current evidence
Buffering	default/custom capacity, partial and full flushes, and sample count
Queries	monotone curves, NaN query probability, exact endpoints, and tensor/cell paths
Discrete data	retained tie intervals, constants, and deterministic signed-zero extrema
Exceptional values	protected positive and negative infinities
Encoding	ordered support, active masses summing to 65,535, and ties-to-even quantization
Accuracy/memory	<code>backend_accuracy</code> representative and adversarial suites with allocator instrumentation
Throughput	Divan benchmarks for normal and uniform tensors at two representative sizes

9.2 Open work

- define and test an atomic NaN policy, or make the NaN-free precondition part of the stable contract;
- test requantization drift through at least one million updates and repeated shifts;
- check in reproducible current K32 accuracy and benchmark result artifacts with platform metadata;
- replace the `RankKnot` and `Quantile Spine merge_*` panic stubs with implementations and fidelity tests;
- add `RankKnot` state export or serialization before claiming durable deferred analysis;
- build visualization, PTQ, or threshold consumers and measure application outcomes;
- continue throughput work on smaller repeated-flush tensors, where K32 still trails t-digest locally.

Ascending streams, blocked modes, and crafted float patterns remain useful diagnostics rather than universal gates. Fixed memory does not provide a worst-case distribution-free accuracy guarantee.

10 Risks and limitations

- **Approximation, not archival:** individual observations and temporal order are irrecoverable, and RankKnot has no snapshot format.
- **Representative data:** no summary can infer a deployment regime that was never observed.
- **Marginal state:** cross-position covariance and sample identity are absent.
- **Requantization drift:** 16-bit masses compactly encode 32 locations but repeatedly approximate old probability.
- **Shared count and NaN:** NaN currently reaches the buffer and panics during flush. Per-position missingness or atomic rejection requires revised semantics or extra work.
- **Tail and mode parity:** no backend wins every workload; sparse zero activations, extreme tails, and separated modes remain important counterexamples.
- **Working memory:** the 208-byte title names state only. The default input buffer raises tensor-scaled retained storage to about 1,232 bytes per f32 position, and worker scratch raises peak heap.
- **Unavailable consumers:** merge, analysis, zero-filtering, serialization, and visualization methods are contract stubs that panic for RankKnot; knots are crate-private; PTQ and direct support export are absent.
- **Fixed memory:** no 208-byte state offers tiny distribution-free error for every stream.

The design principle Collection should preserve a useful approximation of the observed distribution while making as few downstream choices as possible. RankKnot spends its state budget on rank mass, ties, and extrema; everything else is deferred.

11 Reproducibility

Current K32 Rust behavior is defined and exercised by:

- `monatq/src/kernels/rankknot.rs`
- `monatq/tests/rankknot.rs`
- `monatq/src/bin/backend_accuracy.rs`
- `monatq/benches/tensor_digest.rs`

Run `cargo run -p monatq --release --bin backend_accuracy` for the current accuracy/memory report and `cargo bench -p monatq --bench tensor_digest` for throughput. The numeric K32 results in this note came from a local Apple M4 run and are not yet checked in as a machine-readable artifact.

Historical K64 feasibility evidence remains in:

- `docs/experiments/transport_coreset_feasibility.py`
- `docs/experiments/results/transport-coreset-holdout.csv`
- `docs/experiments/README.md`

That script simulates 48-location u32 and 64-location u16 layouts and invokes repository backends through a temporary Rust crate. Its unqualified “RankKnot” label means the historical K64 prototype, not current K32.

12 Conclusion

Current RankKnot is a working K32, f32-only quantile backend. Its 208-byte per-position state preserves exact extrema and retained pure ties, its default live allocation is substantially smaller than both comparison backends, and current Rust accuracy is generally stronger than t-digest on the representative suite without being uniformly best. Local throughput beats t-digest on the larger tested tensor and still trails it on smaller repeated-flush cases.

The broader K64 Python results remain useful evidence for the design’s capacity trade-off, but they are not current-backend results. Merge, analysis, zero-filtering, serialization, and visualization are present in the generic contract but remain panic stubs for RankKnot; direct knot access, clipping search, and quantization are absent. The next decisions should be driven by long-stream drift, reproducible cross-platform throughput, an explicit NaN contract, and real consumer studies rather than by the historical prototype alone.

References

- [1] Z. Karnin, K. Lang, and E. Liberty, “Optimal Quantile Approximation in Streams,” in *2016 IEEE 57th Annual Symposium on Foundations of Computer Science*, 2016, pp. 71–78. doi: [10.1109/FOCS.2016.17](https://doi.org/10.1109/FOCS.2016.17).
- [2] G. Peyré and M. Cuturi, *Computational Optimal Transport*, vol. 11, no. 5–6. Now Publishers, 2019. doi: [10.1561/22000000073](https://doi.org/10.1561/22000000073).
- [3] T. Dunning and O. Ertl, “Computing Extremely Accurate Quantiles Using t-Digests,” *arXiv preprint arXiv:1902.04023*, 2019, doi: [10.48550/arXiv.1902.04023](https://doi.org/10.48550/arXiv.1902.04023).
- [4] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, and D. Soudry, “Accurate Post Training Quantization With Small Calibration Sets,” in *Proceedings of the 38th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 4466–4475. [Online]. Available: <https://proceedings.mlr.press/v139/hubara21a.html>
- [5] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A White Paper on Neural Network Quantization,” *arXiv preprint arXiv:2106.08295*, 2021, doi: [10.48550/arXiv.2106.08295](https://doi.org/10.48550/arXiv.2106.08295).
- [6] R. M. Gray and D. L. Neuhoff, “Quantization,” *IEEE Transactions on Information Theory*, vol. 44, no. 6, pp. 2325–2383, 1998, doi: [10.1109/18.720541](https://doi.org/10.1109/18.720541).
- [7] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [8] R. J. Hyndman and Y. Fan, “Sample Quantiles in Statistical Packages,” *The American Statistician*, vol. 50, no. 4, pp. 361–365, 1996, doi: [10.1080/00031305.1996.10473566](https://doi.org/10.1080/00031305.1996.10473566).